R-107, First Floor, Metro Tower,
Vijay Nagar, Indore (M.P.) 452010

Advanced Agentic AI Systems — Practical Course Curriculum

Cypher Analytica — Design, coordinate, and operate intelligent AI systems

This course advances learners from building individual AI agents to developing systems that manage complex tasks, maintain progress, coordinate specialised components, and recover from failures.

Core concept: Agentic AI describes systems that can pursue goals through tool use, planning, and adaptive execution. Such a system may use **one agent or multiple agents**; multiple agents are useful only when their coordination provides a clear benefit.

Recommended duration: 60 hours of guided training + 30 hours of assignments and project work.

Audience: AI developers, Python programmers, data professionals, and learners who have completed AI Agent Development.
Prerequisites: Python, APIs, JSON, tool calling, retrieval, state management, and basic agent evaluation.

By the end, learners should be able to design and evaluate an agentic application with persistent state, controlled actions, failure recovery, and operational monitoring.

Module-wise curriculum

Module	Hours	Topics covered	Practical activity
1. Agentic System Architecture	4	Single-agent versus multi-agent design; deterministic and agent-controlled steps; responsibilities; system boundaries; autonomy levels; architecture trade-offs	Design an architecture for a research or institute operations assistant
2. Planning and Task Decomposition	5	Breaking goals into tasks; dependencies; execution plans; adapting to new information; completion criteria; budgets; avoiding unnecessary planning	Build an agent that decomposes a research request and updates its plan when information is missing
3. State Machines and Graph-Based Execution	6	States; transitions; conditional routes; branching; parallel execution; shared state;	Implement a research → review → revision process

Module	Hours	Topics covered	Practical activity
		terminating loops; explicit control flow	with a maximum revision limit
4. Durable Execution and Recovery	5	Checkpoints; persistence; pause and resume; retries; timeouts; idempotency; recovery after interruption; compensating actions	Resume an interrupted workflow without repeating a completed external action
5. Multi-Agent Coordination	6	Specialist roles; coordinator–worker patterns; handoffs; sequential and parallel work; task ownership; message contracts; conflicting outputs; coordination overhead	Coordinate research, analysis, and report-writing agents and compare results against a single-agent baseline
6. Memory and Knowledge Management	5	Working memory; persistent memory; retrieval; provenance; freshness; session isolation; conflicting information; retention policies	Maintain task history and retrieve approved information with source references
7. Tool Integration and MCP Foundations	5	Tool contracts; API wrappers; Model Context Protocol concepts; client–server interaction; tool discovery; authentication boundaries; permissions; validating tool results	Connect an agent to a controlled tool service exposing course search and timetable lookup
8. Human Oversight and Security	5	Approval gates; action previews; escalation; prompt injection; untrusted documents; least privilege; sensitive data; audit records; emergency stops	Pause a proposed booking for approval and reject instructions embedded in retrieved content
9. Evaluation and Observability	6	End-to-end task success; tool correctness; groundedness;	Evaluate alternative architectures on the

Module	Hours	Topics covered	Practical activity
		regression tests; execution traces; latency; token and API costs; error analysis; baseline comparisons	same test set and explain their trade-offs
10. Deployment and Capstone Project	13	Application packaging; configuration; queues and concurrency concepts; monitoring; release checks; rollback; documentation; demonstrations	Deploy an agentic application with persistent state, approval gates, and an evaluation report
Total	**60**		

Practical assignments

1. **Adaptive planning:** Build a system that revises its plan when a required source is unavailable.
2. **Graph-based execution:** Implement conditional routes, revision loops, and explicit stopping rules.
3. **Persistent task execution:** Pause a task and resume it from saved state.
4. **Agent coordination:** Delegate independent subtasks and combine outputs using a defined format.
5. **Approval-controlled action:** Prepare an external action, obtain approval, and prevent duplicate execution.
6. **Comparative evaluation:** Compare a fixed workflow, a single agent, and a multi-agent system on a bounded task.

Capstone: Cypher Analytica Learning and Counselling Assistant

Scenario: A learner asks for a suitable learning pathway and a counselling appointment. The system gathers requirements, retrieves approved course information, prepares a grounded recommendation, and manages an approved follow-up.

Start with a single-agent implementation. Introduce specialised agents only where evaluation demonstrates an advantage.

Component	Responsibility
Coordinator	Track the learner's request, identify missing details, and manage task progress
Course knowledge component	Retrieve approved prerequisites, syllabus details, and course information
Learning pathway component	Suggest a sequence based on the learner's goals and existing knowledge
Review component	Check recommendations against retrieved evidence and flag unsupported claims
Scheduling component	Retrieve available slots and prepare a booking
Human approval stage	Confirm the proposed booking or outgoing communication
State and audit store	Record progress, approvals, tool results, and completion status

Required system behavior:

- Ask for clarification when learner requirements are incomplete.
- Support recommendations with approved source information.
- Escalate missing or conflicting course details.
- Request approval before booking or sending messages.
- Recover from interruptions without duplicating actions.
- Stop when the task is complete or its execution budget is reached.
- Keep different learners' records and sessions separate.

Use synthetic learner data during training.

Capstone deliverables

- Architecture diagram and justification for the chosen design.
- Working application with documented tools and permissions.
- Persistent task state and demonstrated pause/resume behaviour.

- Approval interface and audit trail.
- Evaluation dataset covering normal, ambiguous, adversarial, and failure scenarios.
- Report comparing task quality, latency, and cost against a simpler baseline.
- Deployment instructions, operating guide, and final demonstration.

Assessment and certification

Assessment	Weight
Practical assignments	25%
Architecture and design review	15%
Evaluation and failure-recovery testing	20%
Capstone implementation	30%
Demonstration and viva	10%